How Claude Code rules actually work

A practical guide to memory, scoped rules, and predictable behaviour in real projects using Claude Code.

JOSE PARREÑO GARCIA
MAR 25, 2026 • PAID

5 2 Share

This week, I spent time setting up **Claude Code rules** for a personal project.

It seemed straightforward at first. I added a few rules, organised them neatly, and expected Claude to behave more predictably as the project grew. Instead, I started seeing something familiar: behaviour that felt correct in one moment and subtly wrong in the next.

The more I looked into it, the clearer the pattern became. The problem wasn't Claude. It was my mental model of how **Claude Code memory and rules** actually work. I was treating rules like instructions Claude would "remember", rather than context that is discovered, loaded, scoped, and sometimes ignored.

That realisation sent me down a rabbit hole. Why are some rules always loaded? Why do others only apply in certain directories? How do scoped rules actually interact with `CLAUDE.md`? And how can you tell what Claude has *really* loaded into memory, instead of assuming?

This post is my attempt to answer those questions. I break down what Claude Code rules are, how Claude discovers and prioritises them, how scoped rules behave in practice, and what this looks like in a real data science project.

If you are using Claude Code on anything beyond a small experiment, understanding this mental model will save you a lot of confusion.

What this blog will cover

- **What Claude Code rules are, and when not to use them**
  A clear explanation of what rules actually do inside Claude Code, how they differ from prompts or chat instructions, and the situations where rules create more friction than value.
- **How Claude discovers, loads, and prioritises rules**
  How `.claude/rules/` is scanned, how user-level and project-level rules

interact, and how to verify what Claude has actually loaded instead of trusting assumptions.

- **How to use scoped rules deterministically**
  How path-scoped rules work in theory and in practice, how glob patterns behave, and how to design rules that only activate when they genuinely should. In addition, using `CLAUDE.md` as an index and workflow guide
- **How do you tell Claude to read context that is not loaded automatically.**
  In many projects, you will have external documentation you would like Claude to have access to. Claude doesn't load it by default, but there is a way to tell it do so.
- **How can you check what Claude is loading in memory.**
  Never assume that Claude is loading what you are telling it to load. Ask Claude to tell you what it loaded to memory.
- **What this looks like in a real data science project**
  A concrete churn prediction example, showing how to structure rules for notebooks, feature engineering, model training, and production pipelines.
- **What can go wrong (and how to notice early)**
  Common pitfalls such as rule bloat, confusing scoping, and silent failures. Also, some ideas on how to validate behaviour using small, reproducible checks.
- **Closing thoughts, why Claude Code rules exist at all**
  Why rules are a response to very real scaling problems once `CLAUDE.md` files

grow beyond a few dozen lines.

Let's get started!

What Claude Code rules are

In my previous post, I went deep into how Claude Code memory layers work, and why the **global and the project** `CLAUDE.md` files matter.

The global `CLAUDE.md` file has a very specific job: defining your *working relationship* with Claude. One of the best practices mentioned that the global `CLAUDE.md` file should ***not*** describe every possible working preference. It is not the place to explain how to build a churn model, write an executive summary, or run deep research.

Those are project-level nuances and they change with context, and they belong to the project `CLAUDE.md` file. Now, here is where most people get things wrong. Because Claude has this specific project file, there is a temptation of writing every possible project detail here.

- Project context
- Architecture notes
- Coding conventions

- Data constraints
- ML workflows
- Evaluation rules
- Deployment warnings test…

And, to an extent this is correct.

But, if you do that, you might end up with a 1,000 line long chapter of the project description. There are many many problems that branch from such long project files: context memory bloat, contradicting instructions, difficult maintenance and many more (don't worry we will cover details on these later).

The long project file becomes a liability rather than a guide.

Claude Code rules come in to provide structure.

Rules are **project-scoped memory**, stored as small, modular Markdown files under `.claude/rules/`. They exist to capture *how work should be done in this repository*, but in a way that is structured, composable, and maintainable.

For example, we could have a repo will some rule markdown files like the ones below:

```
.claude/rules/
├── python-style.md
├── experiments.md
├── data-privacy.md
└── ml/
    ├── training.md
    └── pipelines.md
```

You might be thinking, *"well, can't I just write these inside the project* `CLAUDE.md` *file?"*. I get the question, only if you think that Claude loads all rules at once. This is not entirely true. We will cover how to help Claude load context rules more intelligently later.

For now, just stick with my deliberately high level summary:

> *At a high level, you can think of Claude Code rules as a way of decomposing project memory. Instead of one large file that tries to explain everything, you get smaller, topic-specific files that can evolve independently.*

That is what the next section is about.

How Claude discovers, loads, and prioritises rules

Claude rules are much more nuanced and powerful than this, so, if you are going to rely on rules, you need to understand **how Claude actually finds them, loads them, and decides which ones apply**.

So, let's start by asking some simple questions:

- If I create a `.claude/rules/` directory with 5 rule files in it, what actually happens?
- Does Claude load all of them at the start of every session?
- Does it wait and load them intelligently when they are needed?
- Or does it do something in between?

Well, the answer is pretty clear.

By default, Claude loads all project rules at the start of a session.

If you have 5 Markdown files under `.claude/rules/`, Claude will see all 5 as soon as it starts working in that repository. *Everything* is in memory.

This means that, without doing any tricks, claude rules are only useful to us humans as an organisational aid. If you are careless with the rules, you can still end up with a bloated context memory.

These “loaded by default rules” are called **unconditional rules**.

What unconditional rules are good for

Unconditional rules are… unconditional. If they exist, they are loaded. Period.

So, if they are always loaded, then what are the best use of them?

Unconditional rules work best when they encode constraints that should apply regardless of what you are doing in the repository. Here are some examples that I follow:

1. **Baseline coding and quality standards**.
   Things like formatting, typing expectations, testing requirements, or logging conventions that apply everywhere in the codebase. These are stable and rarely controversial — which is exactly why they belong here.
2. **Non-negotiable constraints**.
   Security requirements, data privacy rules, or compliance constraints that must always be respected, no matter which file or task Claude is working on.

3. **Shared project assumptions**.
   For example: “this repository targets batch workloads, not real-time systems”, or “this project optimises for offline evaluation correctness over iteration speed”. These assumptions shape many downstream decisions, so having them always present avoids subtle drift.

Notice what is missing from this list.

Unconditional rules are not the place for:

- Step-by-step workflows
- Instructions tied to a specific directory
- ML-specific modelling choices
- Experimental or evolving practices

If a rule only applies when touching a particular part of the codebase, it should not be unconditional. Keeping it unconditional just because it is convenient is how context memory gets fuzzy.

So far, Claude loads everything. Next, we will look at how to tell Claude **what not to load until it is actually needed**, and how to make rules truly contextual rather than just organised.

That is where scoped rules come in.

How to use scoped rules deterministically

Up to this point, rules have behaved in a very blunt way. They exist, so, they load.

That is useful for baseline constraints, but it is not enough for real projects. Most repositories contain *very different kinds of work* living side by side: experiments, pipelines, infrastructure, documentation. If all rules apply everywhere, you are back to context bloat.

So, how do you tell Claude **when** a rule should apply, and just as importantly, **when it should stay out of the way**?

How path-scoped rules work

Claude Code supports **scoped rules** through YAML frontmatter at the top of a rule file. The idea is simple. You tell Claude "*this rule only applies when you are working on files that match these paths*".

For example, imagine you have a repository like the one below.

```
churn-model/
├── CLAUDE.md
├── .claude/
│   └── rules/
│       ├── python-style.md
│       ├── experiments.md
│       ├── data-privacy.md
│       ├── notebooks.md.
│       └── production-ml.md
├── notebooks/
│   ├── 01_eda.ipynb
│   └── 02_feature_ideas.ipynb
├── src/
│   ├── features/
│   │   ├── build_features.py
│   │   └── feature_catalogue.yml
│   ├── models/
│   │   └── churn/
│   │       ├── train.py
│   │       ├── evaluate.py
│   │       └── predict.py
│   └── pipelines/
│       ├── training_pipeline.py
│       └── batch_scoring.py
```

```
└── tests/
    └── test_models_churn.py
```

Now, we want different rules depending on what Claude is touching:

- Notebook work should be lightweight, fast, and explicit about “prototype” status.
- Production model code should be strict: tests, reproducibility, no data leakage.

So, as part of the markdown rules for `notebooks.md` and `production-ml.md`, we would include a `path` to guide Claude.

```
This is the .claude/rules/notebooks.md file.

By adding paths, this tells Claude:
-> When you are working inside notebooks/, Claude sees this rule.
-> When you are editing src/models/..., it should not apply.

---
paths:
  - "notebooks/**"
---
# Notebook rules
```

```
- Treat notebooks as exploratory, not production.
- Prefer small, runnable cells and short outputs.
- If code becomes stable, migrate it into `src/` with tests.
- Do not hardcode secrets or dataset paths; use environment variables or config.
```

```
This is the .claude/rules/production-ml.md file

Similar to the above, these constraints only appear when Claude is touching production
code paths (models and pipeline folders).

---
paths:
  - "src/models/churn/**"
  - "src/pipelines/**"
---
# Production ML rules (churn)

- Any change must preserve reproducibility:
  - fixed random seeds
  - deterministic splits
  - versioned features
- Never train on labels that leak future information.
- Before finalising changes:
  - run `pytest -q`
```

```
  - run the relevant training pipeline locally (or in CI if configured)
- Always report evaluation using the project’s standard metrics (see `experiments.md`).
```

The paths in the rules above are defined via 'glob patterns’. They are pretty easy to define (if you haven’t worked with them, just ask Claude to generate them for you).

Help Claude explore your rules with an index

Now, whilst scoped rules give you a lot of control, they only trigger *if* Claude happens to need to read or work with the paths you specified. What if we could tell Claude what our rules are about to help it understand the project context better?

This is where a simple index can help. A good project `CLAUDE.md` might say things like.

```
## How to work in this repository

This project contains multiple types of work. Before starting, clarify what kind of
task you are doing and follow the relevant guidelines below.

### Exploratory and experimental work

If you are exploring data, prototyping features, or testing modelling ideas:
```

```
- Follow the experimentation guidelines in `.claude/rules/experiments.md`
- Prefer notebooks under `notebooks/`
- Expect looser constraints and faster iteration

### Production ML changes

If you are modifying production model code:

- Expect stricter constraints to apply
- Scoped rules under `src/models/` and `src/pipelines/` will be active
- Reproducibility, evaluation, and testing are mandatory
```

None of that is enforceable on its own. And that is fine.

What it does is prime Claude's planning step. This is where determinism matters, but in a different way.

> Scoped rules are deterministic in *application*.
> `CLAUDE.md` is deterministic in *expectation*.

Another way to see the split:

> Scoped rules prevent **local mistakes**
> `CLAUDE.md` prevents **global misunderstandings**

A couple of examples below:

- If you rely only on scoped rules, Claude might technically obey constraints while still approaching the task in the wrong way. For example, diving into production code when you wanted a design discussion, or optimising code when you wanted an experiment.
- If you rely only on `CLAUDE.md`, Claude might understand intent but miss critical guardrails once it starts editing.
- Used together however, they close the loop.

Up to now, we have focused on memory that Claude loads automatically: global files, project files, and rules. But real projects often have important context that does *not* live in `CLAUDE.md` or `.claude/rules/`

How do you tell Claude to read context that is not loaded automatically?

Real projects almost always have important context that does **not** live in those places.

Think about files like:

- `README.md`
- Architecture overviews
- Design documents, RFCs, PRDs
- Experiment summaries
- Runbooks or operational notes

Claude will not load these by default. Unless you explicitly point to them, Claude may never read them at all.

Explicit context loading with @ imports

Claude Code provides a simple but powerful mechanism for this: **`@` imports**.

When you write `@path/to/file`, you are telling Claude to explicitly load that file into the current session context. There is no inference involved. You are not hinting. You are instructing.

In other words: *"Before you answer or act, read this."*

This context is **task-scoped**, not persistent. It applies only to the current session. That is precisely why it is useful. Here is a relevant example.

```
# Project overview

This repository contains a churn prediction system with:
- exploratory notebooks,
- production training and scoring pipelines,
- and supporting feature engineering code.

For this session, we are working on improving the churn model.

# Relevant context for this task

Before making any changes, please read:
@README.md
@docs/churn_model_design.md
```

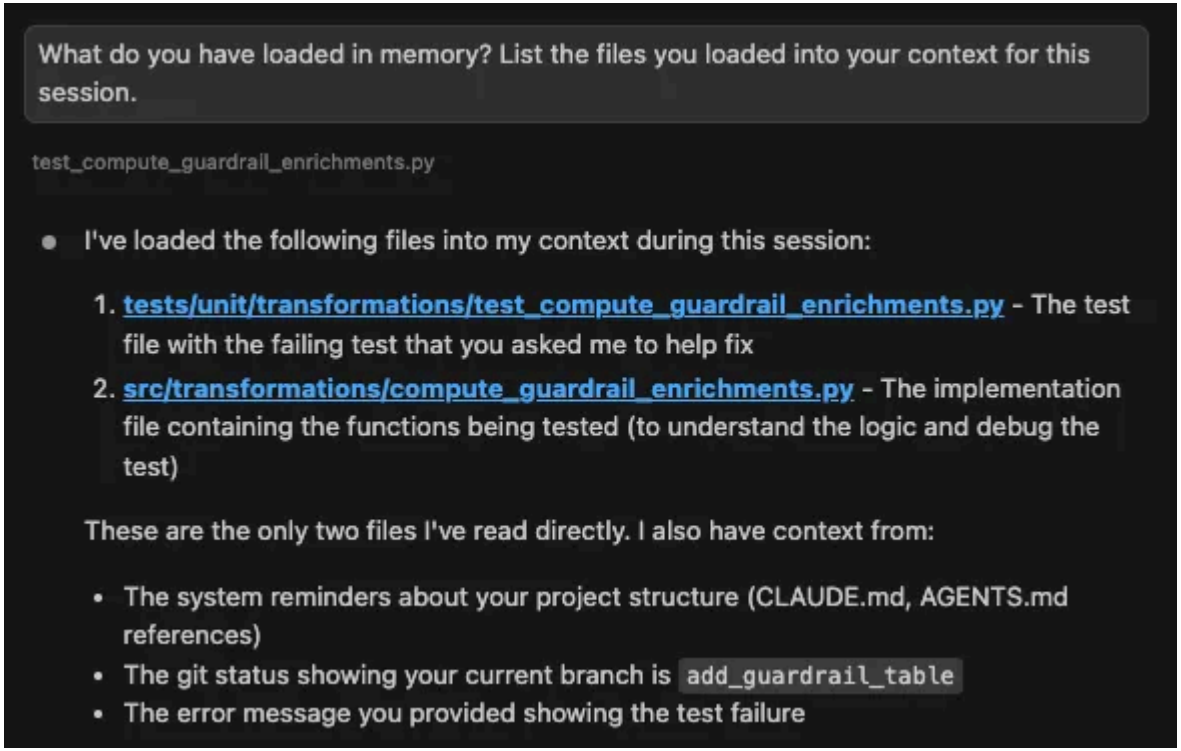
How can you check what Claude is loading in memory.

At this point, we might be tempted to think that we actually know how to control Claude memory. Well, one word of caution: don’t assume too much. In fact, never assume that Claude is loading what you think it is loading. Always inspect.

Ideally, there would be a command that tells you exactly what Claude has loaded into its session context. Today, that command does not exist. The `/memory` command is a memory *management* tool, not a diagnostic one.

Fortunately, there is still a practical workaround: **you can ask Claude directly**.

In the example below, I explicitly asked Claude what it had loaded into memory for the current session. It responded by listing the files it had read directly, along with additional context such as system files, the current Git branch, and the error message I had provided.



There is no interpretation layer here. If a file appears in this list, it is influencing behaviour. If it does not, it is not.

This is currently the most reliable way to reason about what Claude is actually seeing, and it reinforces an important habit: treat memory behaviour as something to be *verified*, not inferred.

What this looks like in a real data science project

Imagine a fairly typical data science repository: a churn prediction system that started with notebooks, grew into feature pipelines, and now has a production training and scoring setup.

The important point is not the exact structure, but the *shape* of the work.

You have, at the same time:

- exploratory notebooks,
- feature engineering code,
- model training and evaluation logic,
- and production pipelines that actually run.

Each of these requires Claude to behave slightly differently.

Repository structure

At a high level, the repository might look like this:

```
churn-model/
├── CLAUDE.md
├── .claude/
│   └── rules/
│       ├── python-style.md
│       ├── experiments.md
│       ├── data-privacy.md
│       ├── notebooks.md
│       └── production-ml.md
├── notebooks/
├── src/
│   ├── features/
│   ├── models/
│   │   └── churn/
│   └── pipelines/
└── tests/
```

Exploratory phase: notebooks and experiments

Early on, you are mostly exploring.

You are in `notebooks/`, testing feature ideas, sanity-checking distributions, iterating quickly. You want Claude to help you reason, prototype, and move fast — not to

lecture you about production constraints. This is where the `notebooks.md` rule and the `experiments.md` rule do their job.

You do not need to remind Claude of this every session. The rules load automatically when you are in the right place. This is already a win.

Transition phase: feature engineering and modelling

As soon as work moves into `src/features/` or `src/models/churn/`, the tone changes. Now:

- Reproducibility matters.
- Data leakage is a risk.
- Evaluation choices shape decision making.

Because these directories are covered by scoped rules, Claude switches mode automatically. You did not change *how* you prompted Claude. You changed *where you were working*.

Where `CLAUDE.md` fits in day to day

Now imagine you start a new session and say something like:

> "I want to improve the churn model."

Without guidance, Claude could reasonably:

- jump into notebooks,
- refactor production code,
- or start writing new pipelines.

This is where the project `CLAUDE.md` acts as an index.

When explicit context is needed

Finally, there are moments where neither rules nor indexing are enough. For example:

- reviewing an architecture decision,
- revisiting an old experiment,
- aligning on business assumptions.

That is when you explicitly pull in context with `@` imports:

What can go wrong (and how to notice early)

By now, Claude Code rules might sound like a clean, almost mechanical system. In practice, there are a few ways things tend to go wrong.

The most common one is rule bloat.

As soon as rules prove useful, there is a temptation to add more of them. The result looks modular, but behaves like a bloated `CLAUDE.md` split across files. Not only the context window can be bloated by unnecessary tokens, but most probably, **you will have rules that contradict each other.**

Another common pitfall is confusing or unstable scoping.

Path-scoped rules only work as well as the boundaries you choose. If directories move frequently, or if scopes rely on fragile glob patterns, rules may stop applying. Claude does not warn you when this happens.

Finally, there are silent failures.

Claude might appear to "ignore" a rule when in reality the rule was never loaded, never scoped correctly, or overridden by context you did not realise was active. This is where most frustration comes from.

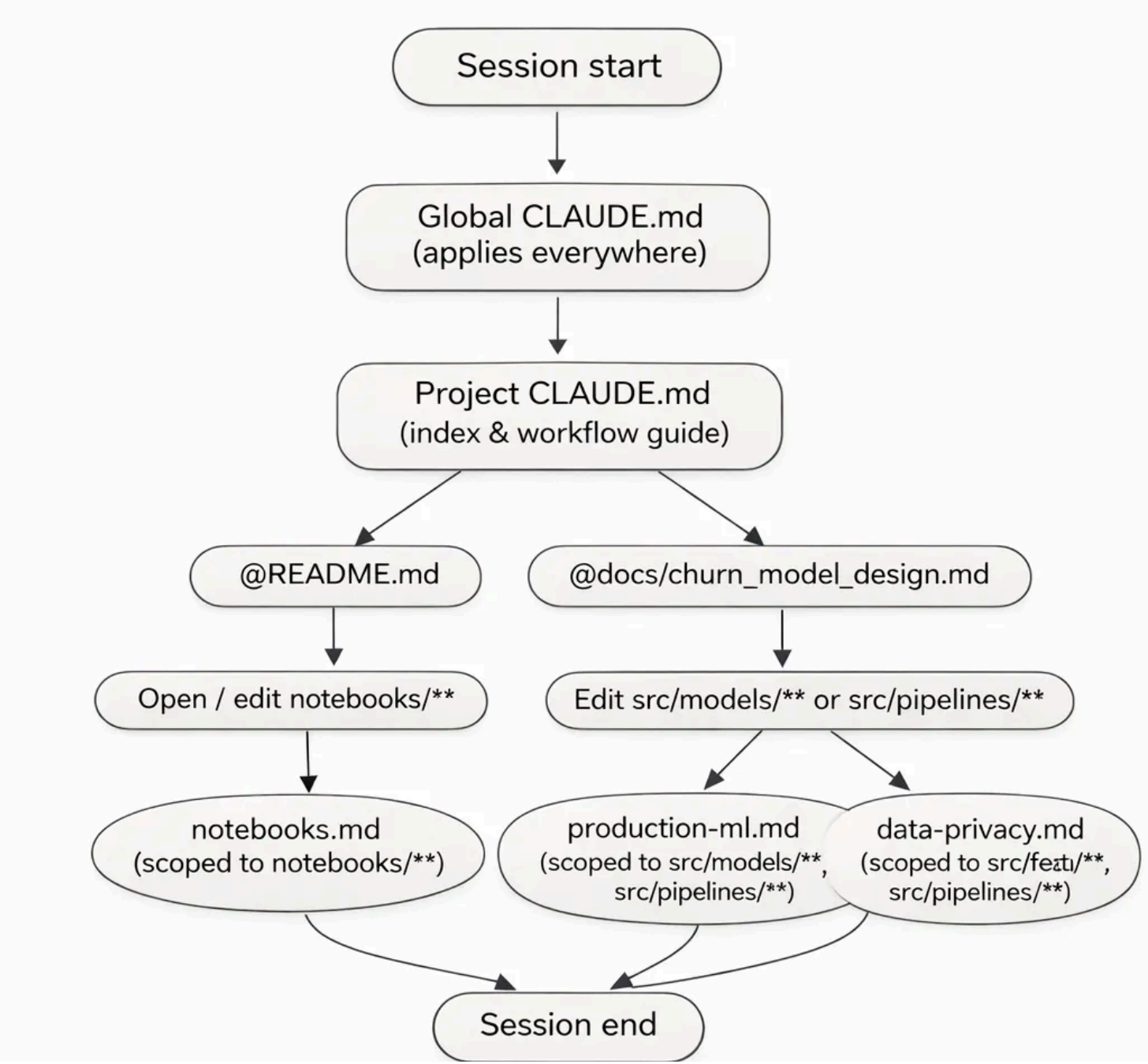
The antidote to all of this is surprisingly simple: small, reproducible checks.

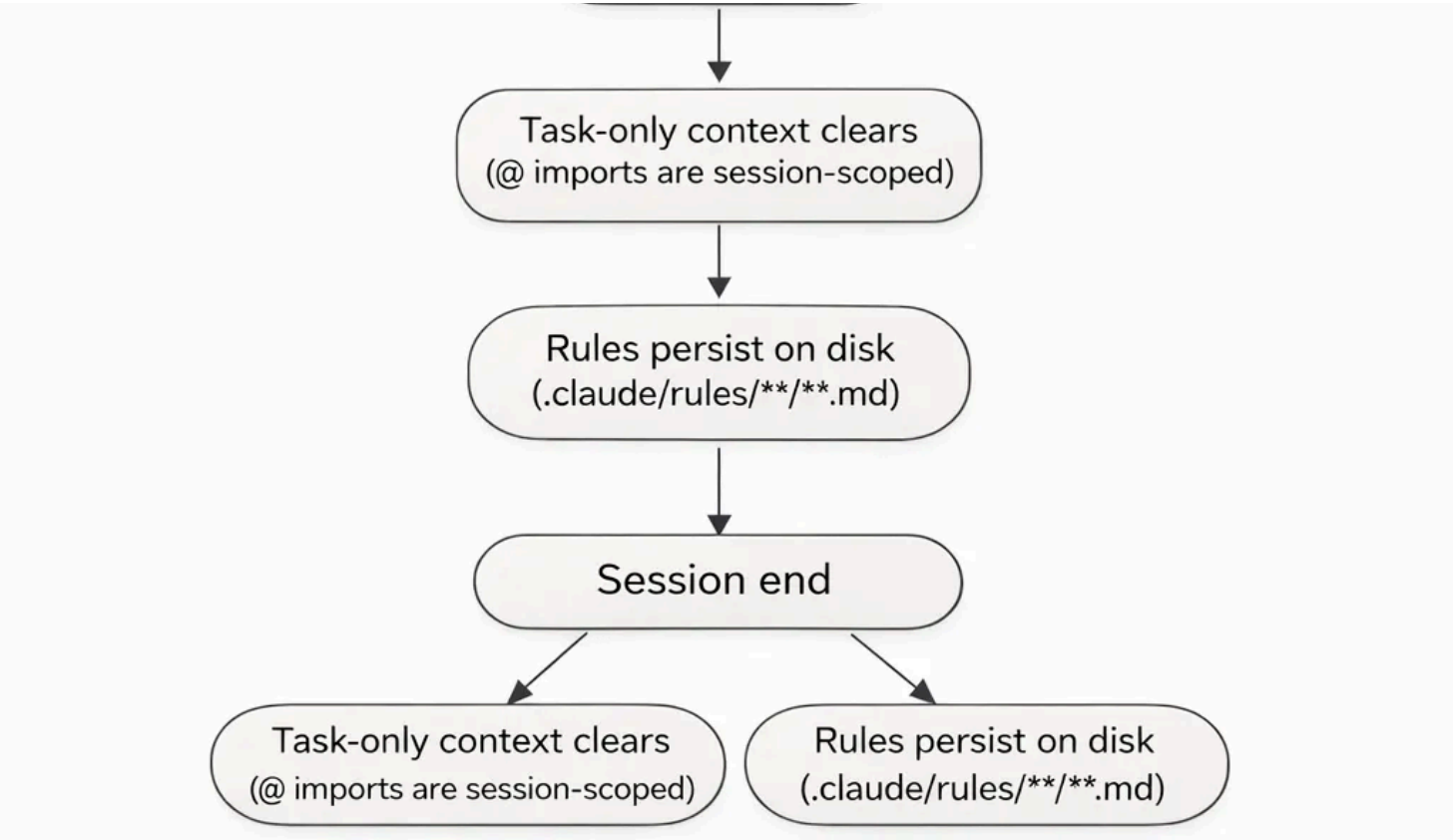
When you add or change a rule:

- open a file that should trigger it and observe behaviour,

- open a file that should not and observe the difference,
- ask Claude explicitly what context it has loaded for the task.

If behaviour surprises you, treat it as a debugging problem, not an intelligence problem. When you can explain why Claude behaved the way it did, the system is working.


Session start
Global CLAUDE.md
(applies everywhere)
Project CLAUDE.md
(index & workflow guide)
@README.md
@docs/churn_model_design.md
Open / edit notebooks/**
Edit src/models/** or src/pipelines/**
notebooks.md
(scoped to notebooks/**)
production-ml.md
(scoped to src/models/**,
src/pipelines/**)
data-privacy.md
(scoped to src/feat/**,
src/pipelines/**)
Session end



Closing thoughts, why Claude Code rules exist at all

I see Claude Code rules as a response to a very mundane problem: **projects grow**.

What starts as a tidy `CLAUDE.md` quickly becomes a dumping ground. Style preferences, workflows, warnings, edge cases, and historical decisions all accumulate

in one place. At some point, the file stops being a guide and starts being a liability.

Rules exist to break that pattern.

They let you decompose project memory into smaller, topic-specific pieces. They give you a way to say “this always applies”, “this only applies here”, and “this only matters for this task”, without repeating yourself in every session.

Dont think rules make Claude smarter. It’s more about making your projects more legible to Claude, to your team, and to your future self.

Interested in some cool community repos with Claude Code rules?

- https://github.com/affaan-m/everything-claude-code
- https://github.com/jarrodwatts/claude-code-config

Now, I want to hear from you!

- Have you already tried using rules or scoped memory in your own projects?
- Did this way of thinking change how you structure `CLAUDE.md` files or project documentation?

- Have you run into cases where rules helped a lot — or where they silently caused confusion?

Drop your thoughts, questions, or war stories in the comments. I am genuinely curious how others are using Claude Code rules in practice 👇

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

5 Likes • 2 Restacks

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture